Hand Swipe Direction Detection App

Development Plan — Based on the System Architecture

Purpose: A simple, beginner-friendly development roadmap for implementing the classroom computer-vision project.

1. Development Goal

Develop a Python-based webcam application that detects whether a user's hand has swiped LEFT, RIGHT, UP, or DOWN by comparing hand positions across multiple video frames. The implementation should remain local, simple, practical, and easy to explain.

The architecture explicitly treats swipe detection as temporal movement detection rather than static hand-position detection. ■filecite■turn1file0■L22-L37■

2. Technology Stack

- Python — main programming language.
- OpenCV (cv2) — webcam access, frame capture, display, and basic image processing.
- NumPy — coordinate and movement calculations.
- MediaPipe — optional, recommended for easier hand landmark detection and tracking.

No custom deep-learning training is required for the core project. The architecture allows MediaPipe as an optional hand-landmark solution. ■filecite■turn1file0■L185-L197■

3. Development Phases

Phase	Main Work	Output
1. Project Setup	Create folders, virtual environment, install libraries	Working project structure
2. Webcam Module	Open camera, read frames, display feed, Q exit	Live webcam feed
3. Hand Detection	Detect one hand and obtain a usable position/landmark	Hand position
4. Hand Tracking	Store previous/current positions across frames	Tracked coordinates
5. Movement Calculation	Calculate ΔX and ΔY	Movement values
6. Swipe Classification	Apply dominance and threshold rules	LEFT/RIGHT/UP/DOWN
7. Confirmation	Add minimum movement/frame checks and optional cooldown	Fewer false swipes
8. UI & Error Handling	Display status/result and handle failures safely	Usable application
9. Testing	Run defined test cases and tune thresholds	Validated system
10. Documentation	README, screenshots, explanation, demo preparation	Final class project

4. Phase 1 — Project Setup

- Create the project directory: hand_swipe_detection/

- Create main.py, hand_detection.py, hand_tracking.py, swipe_detection.py, config.py, requirements.txt, README.md.
- Create utils/drawing.py and a screenshots/ directory.
- Create and activate a Python virtual environment.
- Install the required libraries.

```
hand_swipe_detection/
■■■ main.py
■■■ hand_detection.py
■■■ hand_tracking.py
■■■ swipe_detection.py
■■■ config.py
■■■ requirements.txt
■■■ README.md
■■■ utils/
■   ■■■ drawing.py
■■■ screenshots/
```

This structure follows the architecture's proposed classroom folder organization. ■filecite■turn1file0■L457-L490■

5. Phase 2 — Webcam and Frame Capture

- Initialize the webcam using OpenCV.
- Check whether the camera opens successfully.
- Capture frames continuously.
- Validate each frame before processing.
- Display the live webcam feed.
- Terminate cleanly when Q is pressed.

```
Webcam
   ↓
Camera Initialization
   ↓
Frame Capture
   ↓
Frame Validation
   ↓
Processing Pipeline
```

Do not add hand detection yet. First make sure the camera and frame loop work reliably.

6. Phase 3 — Hand Detection

Choose one implementation approach. For a beginner project, MediaPipe is the simpler route because it can provide hand landmarks directly. The alternative is OpenCV + NumPy using preprocessing, segmentation/thresholding, contours, a bounding box, and a centroid. ■filecite■turn1file0■L199-L227■

- Detect a single primary hand.
- Obtain a stable coordinate representing the hand.
- If using MediaPipe, select one useful landmark or derive a representative hand position.
- If no hand is detected, do not treat it as an application failure.
- Return a clear result such as hand detected / no hand detected.

7. Phase 4 — Hand Tracking

The tracking module answers four questions: where was the hand, where is it now, how far did it move, and which direction did it move? The architecture uses positions such as (x1,y1), (x2,y2), and (x3,y3) over time. ■filecite■turn1file0■L229-L246■

- Store the previous hand position.
- Read the current hand position.

- Update the previous position after each valid frame.
- Reset tracking if the hand disappears for the current movement.

```
Previous Position
       +
Current Position
       ↓
Movement Calculation
```

8. Phase 5 — Movement Calculation

For each valid movement step:

```
ΔX = Current X - Previous X
ΔY = Current Y - Previous Y
```

- Compare |ΔX| and |ΔY|.
- If horizontal movement dominates, classify LEFT or RIGHT.
- If vertical movement dominates, classify UP or DOWN.
- Remember that image Y coordinates normally increase downward, so positive ΔY corresponds to DOWN and negative ΔY corresponds to UP. ■filecite■turn1file0■L249-L270■

9. Phase 6 — Threshold and Swipe Classification

A small movement should not immediately become a swipe. Use a configurable movement threshold.

```
If |ΔX| > |ΔY|:
    ΔX > threshold  → SWIPE RIGHT
    ΔX < -threshold → SWIPE LEFT

If |ΔY| > |ΔX|:
    ΔY > threshold  → SWIPE DOWN
    ΔY < -threshold → SWIPE UP
```

- Keep MOVEMENT_THRESHOLD in config.py.
- Do not assume one universal value; tune it according to camera resolution, distance, hand size, and frame rate.

These threshold considerations are explicitly identified in the architecture. ■filecite■turn1file0■L273-L292■

10. Phase 7 — Swipe Confirmation and Cooldown

- Record the initial position.
- Require movement to continue for an appropriate number of frames or sufficient distance.
- Confirm the swipe only after the movement threshold is exceeded.
- Optionally apply a short cooldown after a confirmed swipe.
- Reset tracking after the swipe so one continuous motion does not produce repeated RIGHT/LEFT/etc. detections.

The architecture proposes minimum movement distance, minimum frames, dominant direction, and optional cooldown as simple confirmation mechanisms. ■filecite■turn1file0■L294-L325■

11. Phase 8 — Display and User Interface

- Show the live webcam feed.
- Show 'No Hand Detected' when no hand is present.
- Show 'Hand Detected' or 'Tracking...' while a movement is being observed.
- Show the final result: SWIPE LEFT, SWIPE RIGHT, SWIPE UP, or SWIPE DOWN.
- Optionally draw a hand landmark/bounding box and movement arrow.
- Keep the interface simple; no advanced GUI is required.

12. Phase 9 — Error Handling

- Webcam unavailable → show a camera error and terminate safely.
- Invalid frame → skip/retry or terminate after repeated failures.
- No hand detected → reset/pause tracking; this is a normal condition.
- Hand lost during swipe → show tracking lost and reset the current movement.
- Invalid coordinates → ignore/reset rather than crashing.
- Multiple hands → use the primary/largest hand or show a warning.
- Unexpected runtime error → use basic exception handling.
- User presses Q → release the camera and close OpenCV windows.

The architecture specifically includes these error conditions and a clean Q-based exit. ■filecite■turn1file0■L350-L387■

13. Phase 10 — Configuration

- MOVEMENT_THRESHOLD — minimum movement required.
- MIN_FRAMES — minimum frames used for confirmation.
- COOLDOWN_TIME — optional delay/reset period.
- CAMERA_INDEX — webcam selection.
- FRAME_WIDTH and FRAME_HEIGHT — camera resolution.

Keep configurable values outside the main application logic. ■filecite■turn1file0■L493-L501■

14. Recommended Module Responsibilities

File	Responsibility
main.py	Controls the main webcam/application loop.
hand_detection.py	Detects the hand and returns usable position/landmark information.
hand_tracking.py	Maintains previous/current hand positions.
swipe_detection.py	Calculates ΔX/ΔY, checks threshold, and classifies direction.
config.py	Stores adjustable thresholds and camera settings.
utils/drawing.py	Draws landmarks, boxes, arrows, and status text.
requirements.txt	Lists project dependencies.
README.md	Explains setup, usage, architecture, and testing.

15. Core Development Flow

```
main.py
   ↓
Open Webcam
   ↓
Read Frame
   ↓
Validate Frame
   ↓
Detect Hand
   ↓
Get Hand Position
   ↓
Track Previous + Current Position
   ↓
Calculate ΔX / ΔY
   ↓
Check Movement Threshold
```

```
   ↓
Confirm Swipe
   ↓
Classify Direction
   ↓
Display Result
   ↓
Repeat until Q
```

This directly follows the architecture's complete data flow from webcam input through hand detection, tracking, movement calculation, threshold checking, classification, and display. ■filecite■turn1file0■L429-L455■

16. Testing and Validation

ID	Test	Expected Result
TC01	Webcam unavailable	Camera Error
TC02	Webcam available	Webcam Feed
TC03	No hand	No Hand Detected
TC04	Hand moves left	SWIPE LEFT
TC05	Hand moves right	SWIPE RIGHT
TC06	Hand moves up	SWIPE UP
TC07	Hand moves down	SWIPE DOWN
TC08	Very small movement	No Swipe
TC09	Hand disappears	Tracking Lost
TC10	Multiple hands	Primary Hand / Warning
TC11	Invalid frame	Handled safely
TC12	Press Q	Clean Exit

These test cases are taken from the architecture document. ■filecite■turn1file0■L531-L548■

17. Threshold Tuning Procedure

- Start with a conservative threshold.
- Test slow LEFT, RIGHT, UP, and DOWN movements.
- Test small movements that should not trigger a swipe.
- Test faster movements.
- Test at different distances from the webcam.
- Adjust MOVEMENT_THRESHOLD and MIN_FRAMES until normal hand motion produces reliable results.
- Record the final configuration values in README.md.

18. Optional API / LLM Stage

Do not develop this before the core detector works. The architecture explicitly keeps API/LLM functionality outside the core system.

```
Core:
Webcam → Hand Detection → Tracking → Swipe Detection

Optional:
Swipe Result → API/LLM → Text or Action
```

- First complete all four swipe directions locally.
- Only then consider adding an optional response such as 'User swiped to the right.'

The core application is intended to work without internet access or an external API. ■filecite■turn1file0■L503-L528■

19. Final Development Checklist

- Webcam opens and closes correctly.
- Frames are captured and validated.
- One primary hand can be detected.
- Hand position is tracked across frames.
- Previous and current coordinates are maintained.
- ΔX and ΔY are calculated correctly.
- Horizontal/vertical dominance is determined.
- Movement threshold prevents tiny false swipes.
- LEFT, RIGHT, UP, and DOWN are detected.
- Swipe confirmation reduces false detections.
- Optional cooldown prevents repeated detections.
- Results are displayed on the webcam feed.
- No-hand and tracking-lost conditions are handled.
- Invalid frames/coordinates do not crash the program.
- Q performs a clean exit.
- All architecture test cases pass.
- README and screenshots are prepared.

20. Suggested Development Order

For a beginner, implement the project in this exact order so each stage can be tested before adding complexity:

```
1. Webcam → live video
2. Hand detection → hand found/not found
3. Hand position → x,y coordinates
4. Tracking → previous/current coordinates
5. ΔX/ΔY → movement values
6. Threshold → ignore small movement
7. Classification → LEFT/RIGHT/UP/DOWN
8. Confirmation → reduce false detections
9. Cooldown → avoid repeated swipes
10. Display → result on webcam
11. Error handling → safe operation
12. Testing → tune and validate
13. Documentation → README + screenshots
14. Optional API/LLM → only if core system is complete
```

21. Expected Final Result

The completed application should be a small local computer-vision system that reads webcam frames, detects a single hand, tracks its position over time, calculates ΔX/ΔY, checks movement thresholds, classifies the dominant swipe direction, and displays the result. This matches the architecture's final goal of a simple, practical, beginner-friendly classroom project rather than an enterprise system. ■filecite■turn1file0■L666-L685■